

Faculdade de Ciências da Universidade do Porto

Departamento de Matemática

Métodos Numéricos (M2039)

Ano Letivo 2026/2027

Trabalho Prático 1

Relatório

Realizado por:

Pedro André Silva Pinho Gonçalves	up202405642
Filipe Marques Mano	up202506000
Martim Andrade Ribeiro	up202505495

Outubro 2026

1 Motivação do trabalho

Este trabalho prático tem como foco a resolução de problemas de aproximação de constantes matemáticas (como e e π^2) em ambiente computacional, exigindo que os resultados cumpram critérios de paragem analíticos baseados na majoração do erro de truncatura. Paralelamente, procuramos investigar o limite de precisão da máquina (epsilon da máquina) para ver como este pode influenciar os resultados obtidos.

2 Resolução da Questão 1

Para este trabalho, decidimos usar a linguagem python, o que nos facilita muito a vida neste exercício já que a solução é tão simples quanto ler uma constante embutida:

```
import sys
print(sys.float_info.epsilon)
```

Ainda assim, o exercicio pede para "calcular" o valor de epsilon de máquina, e aqui não é bem isso que estamos a fazer. Por este motivo decidimos implementar algumas maneiras alternativas de obter este valor. Uma maneira é encontrar o valor imediatamente maior do que 1 e subtrair 1:

```
import math
print(math.nextafter(1.0, math.inf) - 1.0)
```

Alternativamente, se não quisermos importar módulos, podemos dividir 1 por 2 repetidamente até termos um valor que, ao somar a 1, não altera o valor e retornar o dobro deste (Vale notar que isto só funciona porque valores em vírgula flutuante IEEE 754 realizam arredondamento ao par mais próximo):

```
def epsilon():
    eps = 1.0

    while 1 + eps > 1:
        eps /= 2

    return eps*2

print(epsilon())
```

Todos estes programas imprimem o valor de epsilon de máquina para precisão dupla:

$$2.220446049250313 \times 10^{-16}$$

3 Resolução da Questão 2

3.1 Majorar o erro

No âmbito da resolução da questão, é pedido que seja calculado um valor aproximado da soma da série dada por:

$$S = 18 \sum_{k=1}^{\infty} \frac{k!^2}{k^2(2k)!} \tag{1}$$

Inicialmente, procuramos definir uma simplificação para o termo geral da série S, dado por: $a_k = \frac{k!^2}{k^2(2k)!}$, de modo a obter uma nova expressão através da qual possamos realizar o cálculo aproximado. Para isso, analisamos o termo seguinte, ou seja, $a_{k+1} = \frac{(k+1)!^2}{(k+1)^2(2(k+1))!}$, e realizamos a divisão entre os dois termos, obtendo:

$$\begin{aligned}
\frac{a_{k+1}}{a_k} &= \frac{(k+1)!^2}{(k+1)^2(2(k+1))!} \cdot \frac{k^2(2k)!}{k!^2} \\
&= \frac{(k+1)^2 k!^2}{(k+1)^2(2k+2)(2k+1)(2k)!} \cdot \frac{k^2(2k)!}{k!^2} \\
&= \frac{k^2}{(2k+1)(2k+2)} \\
&= \frac{k^2}{4k^2+6k+2}
\end{aligned}$$

Após esta simplificação, deduzimos que o termo geral da série S pode ser reescrito como:

$$a_{k+1} = a_k \cdot \frac{k^2}{4k^2+6k+2} \tag{2}$$

De modo a majorar o termo geral da série, podemos observar que, como k é um número positivo, temos $4k^2 + 6k + 2 > 4k^2$, o que nos permite escrever a seguinte desigualdade:

$$\frac{k^2}{4k^2+6k+2} < \frac{k^2}{4k^2} = \frac{1}{4} \tag{3}$$

Assim, temos que:

$$\forall k \frac{a_{k+1}}{a_k} < \frac{1}{4} < 1 \tag{4}$$

Logo, podemos usar a seguinte fórmula para majorar o erro:

$$\Delta S_k \leq \frac{a_{k+1}}{1 - \frac{1}{4}} = a_{k+1} \times \frac{4}{3} \tag{5}$$

3.2 Primeiro algoritmo

Inicialmente, decidimos implementar o algoritmo desta maneira:

```
def S(eps):

    # Variaveis
    acc = 0.0
    ter = 9.0
    err = math.inf
    k = 1

    # Calcular e somar termos
    while (err > eps and k < 100):
        acc += ter
        ter *= (k ** 2) / ((2*k + 1) * (2*k + 2))
        err = (4/3) * ter
        k += 1

    # Retornar o numero de iteracoes,
    # o resultado e o erro absoluto
    return k - 1, acc, abs(math.pi**2 - acc)
```

Após executar os testes pedidos no exercicio, obtivemos a seguinte tabela:

ϵ	**Iterações**	**Resultado**	**Erro Absoluto Efetivo**
10^{-8}	13	9.869604398132752	$2.9566056269914043 \times 10^{-}09$
10^{-9}	14	9.869604400421997	$6.673612773511195 \times 10^{-}10$
10^{-10}	16	9.869604401054712	$3.4646063795662485 \times 10^{-}11$
10^{-11}	17	9.869604401081402	$7.956302283673722 \times 10^{-}12$
10^{-12}	19	9.869604401088933	$4.2454928461665986 \times 10^{-}13$
10^{-13}	21	9.869604401089337	$2.1316282072803006 \times 10^{-}14$
10^{-14}	22	9.869604401089354	$3.552713678800501 \times 10^{-}15$
10^{-15}	24	9.86960440108936	$1.7763568394002505 \times 10^{-}15$

Tabela 1: Resultados da aproximação da série S (π^2)

Algo que podemos notar olhando para a tabela é que a função falha no último teste. Era esperado um valor com erro menor do que 10^{-15}, no entanto foi calculado um valor de erro superior. Como π^2 está entre 8 e 16, o seu expoente em representação virgula flutuante binária é igual a 3, e como nós estamos a trabalhar com dupla precisão, temos que o erro por arredondamento é de até $2^{3-53} = 2^{-50} \approx 9 \times 10^{-16}$, o que é muito próximo de 10^{-15}. Isto significa que é muito difícil de obter um erro menor do que 10^{-15}, mas não é impossível.

3.3 Segundo algoritmo

Modificando o programa para, em vez de somar os termos à medida que os calcula, realizar as somas de menor magnitude primeiro de modo a minimizar os erros por arredondamento, obtemos o seguinte código e tabela:

```
def S(eps):

    # Variaveis
    lst = []
    ter = 9.0
    err = math.inf
    k = 1
    res = 0.0

    # Calcular termos
    while (err > eps and k < 25):
        lst.append(ter)
        ter *= (k**2) / ((2*k + 1) * (2*k + 2))
        err = (4/3) * ter
        k += 1

    # Somar em ordem crescente de magnitude
    lst.sort(key = abs)
    while len(lst) > 1:
        res = lst[0] + lst[1]
        lst = lst[2:]
        bisect.insort(lst, res, key = abs)
    res = lst[0]

    # Retornar o numero de iteracoes,
    # o resultado e o erro absoluto
    return (k - 1, res, abs(math.pi**2 - res))
```

ϵ	**Iterações**	**Resultado**	**Erro Absoluto Efetivo**
10^{-8}	13	9.86960439813275	$2.9566074033482437 \times 10^{-}09$
10^{-9}	14	9.869604400421997	$6.673612773511195 \times 10^{-}10$
10^{-10}	16	9.86960440105471	$3.4647840152501885 \times 10^{-}11$
10^{-11}	17	9.8696044010814	$7.958078640513122 \times 10^{-}12$
10^{-12}	19	9.869604401088933	$4.2454928461665986 \times 10^{-}13$
10^{-13}	21	9.869604401089335	$2.3092638912203256 \times 10^{-}14$
10^{-14}	22	9.869604401089353	$5.329070518200751 \times 10^{-}15$
10^{-15}	24	9.869604401089358	0.0

Tabela 2: Resultados da aproximação da série S (π^2)

Se olharmos de novo para o ultimo teste, vemos que o erro absoluto agora é marcado como 0.0, isto significa que o resultado é exatamente igual à representação de π^2 em precisão dupla, ou seja, o erro associado é o erro por arredondamento: $2^{-50} < 9 \times 10^{-16} < 10^{-15}$
Assim, o nosso programa satisfaz todos os critérios do exercicio.

4 Resolução da Questão 3

4.1 Métodos

Para a aproximação da constante de Euler ($e \approx 2.71828...$), o guião sugere a utilização de duas abordagens matemáticas distintas, a Fórmula A e a Fórmula B, definidas respetivamente por:

$$\text{Fórmula A:} \quad a_n = \left(1 + \frac{1}{n}\right)^n$$

$$\text{Fórmula B:} \quad b_m = \sum_{k=0}^{m} \frac{1}{k!}$$

A implementação da Fórmula A consiste na tradução direta da expressão algébrica numa função simples:

```
def a_euler(n):
    return (1 + 1/n)**n>
```

Já na Fórmula B, o algoritmo baseia-se num ciclo `while (k <= m)` que vai acumulando os termos da série a cada iteração. Para otimizar o código, evitamos o cálculo repetitivo de fatoriais do zero a cada iteração implementando uma relação de recorrência. Sabendo que o termo geral da série é $a_k = \frac{1}{k!}$, o termo seguinte pode ser deduzido a partir do termo atual através da expressão:

$$a_{k+1} = \frac{1}{(k+1)!} = \frac{1}{(k+1) \times k!} = a_k \times \frac{1}{k+1}$$

Desta forma, o programa calcula o próximo valor a adicionar à soma multiplicando simplesmente o termo anterior por $\frac{1}{k+1}$:

```
def b_euler(m):
    soma = 0.0
    termo = 1.0
    k = 0
    while (k <= m):
        soma += termo
        termo = termo * (1 / (k + 1))
        k += 1
    return soma
```

4.2 Discussão dos Resultados da Formula A

Pela análise dos resultados da fórmula A (Tabela 3), observa-se que, ao aumentar o valor de n, a sucessão começa a convergir para o valor da constante de Euler. No entanto, a partir de $n = 10^{12}$, nota-se o início de uma divergência progressiva em relação ao valor de referência. Esta alteração deve-se à aproximação ao limite de precisão da máquina, o que provoca perdas severas de algarismos significativos. Ao elevar esta base, já com erros de arredondamento, a um expoente muito elevado, o pequeno erro inicial sofre uma propagação massiva que corrompe o resultado final. O caso extremo ocorre quando $n = 10^{16}$, onde a parcela $1/n$ (10^{-16}) é inferior ao epsilon da máquina. Nesta iteração, o computador não consegue distinguir 1.0 da soma $1.0 + 10^{-16}$, efetuando o arredondamento para 1.0. Por fim, a operação realizada é $1.0^{10^{16}}$, o que resulta em 1.0.

n	**Resultado**	**Erro Absoluto Efetivo**
10^1	2.5937424601000023	1.245×10^{-1}
10^2	2.7048138294215285	1.347×10^{-2}
10^3	2.7169239322355936	1.358×10^{-3}
10^4	2.7181459268249255	1.359×10^{-4}
10^5	2.7182682371922975	1.359×10^{-5}
10^6	2.7182804690957534	1.359×10^{-6}
10^7	2.7182816941320818	1.343×10^{-7}
10^8	2.7182817983473577	3.011×10^{-8}
10^9	2.7182820520115603	2.236×10^{-7}
10^{10}	2.7182820532347876	2.248×10^{-7}
10^{11}	2.7182820533571100	2.249×10^{-7}
10^{12}	2.7185234960372378	2.417×10^{-4}
10^{13}	2.7161100340869010	2.172×10^{-3}
10^{14}	2.7161100340870230	2.172×10^{-3}
10^{15}	3.0350352065492620	3.168×10^{-1}
10^{16}	1.0000000000000000	1.718×10^{0}

Tabela 3: Resultados da aproximação da constante de Euler utilizando a Fórmula A.

4.3 Discussão dos Resultados da Formula B

Por outro lado, a fórmula B (Tabela 4) mostra um comportamento estável. Ao aumentar a quantidade de termos da série, os valores convergem de forma consistente para o valor real da constante, verificando-se uma estagnação a partir da iteração $m = 17$. Esta estagnação deve-se ao limite de precisão do computador. Para $m = 18$, o termo a adicionar à soma é $1/18! \approx 1.56 \times 10^{-16}$, que é inferior ao epsilon da máquina. Por conseguinte, os termos a partir da décima oitava iteração não agregam qualquer valor à soma acumulada, uma vez que estes excedem a capacidade de representação da mantissa do sistema, sendo ignorados durante o processo de arredondamento.

m	**Resultado**	**Erro Absoluto Efetivo**
1	2.0000000000000000	7.183×10^{-1}
2	2.5000000000000000	2.183×10^{-1}
3	2.6666666666666665	5.162×10^{-2}
4	2.7083333333333330	9.948×10^{-3}
5	2.7166666666666663	1.615×10^{-3}
6	2.7180555555555554	2.263×10^{-4}
7	2.7182539682539684	2.786×10^{-5}
8	2.7182787698412700	3.059×10^{-6}
9	2.7182815255731922	3.029×10^{-7}
10	2.7182818011463845	2.731×10^{-8}
11	2.7182818261984930	2.261×10^{-9}
12	2.7182818282861687	1.729×10^{-10}
13	2.7182818284467594	1.229×10^{-11}
14	2.7182818284582300	8.149×10^{-13}
15	2.7182818284589950	5.018×10^{-14}
16	2.7182818284590430	2.220×10^{-15}
17	2.7182818284590455	4.441×10^{-16}
18	2.7182818284590455	4.441×10^{-16}
19	2.7182818284590455	4.441×10^{-16}
20	2.7182818284590455	4.441×10^{-16}

Tabela 4: Resultados da aproximação da constante de Euler utilizando a Fórmula B.